

Projeto SAIRA

Arquitetura de software



manacá
TECNOLOGIAS SOCIAIS

Projeto SAIRA

Arquitetura de software



SUMÁRIO

1. Visão executiva.....	3
2. Diagrama da solução.....	4
3. Detalhamento dos componentes.....	5
a. Camada de ingestão.....	6
b. Fila e desacoplamento.....	6
c. Núcleo de inteligência.....	6
d. Interfaces de usuário.....	7
4. Infraestrutura de armazenamento e dados.....	7
a. Perspectiva futura: armazenamento de vídeo.....	9
5. Stack tecnológico.....	10
a. Camada de software.....	10
b. Especificação do servidor.....	10
6. Observabilidade e monitoramento.....	11
7. Segurança e controle de acesso.....	11
8. Estratégia de implantação (CI/CD).....	12



1. Visão executiva

O objetivo desta arquitetura é viabilizar o monitoramento de descarte irregular de lixo em tempo real utilizando câmeras IP convencionais distribuídas pela cidade de Recife.

A arquitetura da solução foi desenhada com foco na agilidade de implementação, visando colocar o sistema em operação e demonstrar valor para a gestão pública o mais rápido possível. Ao optar inicialmente pelo processamento de inteligência artificial centralizado na nuvem (AWS) conectado a câmeras IP convencionais, a solução reduz a complexidade de hardware na ponta, permitindo uma validação acelerada do modelo tecnológico e de negócio.

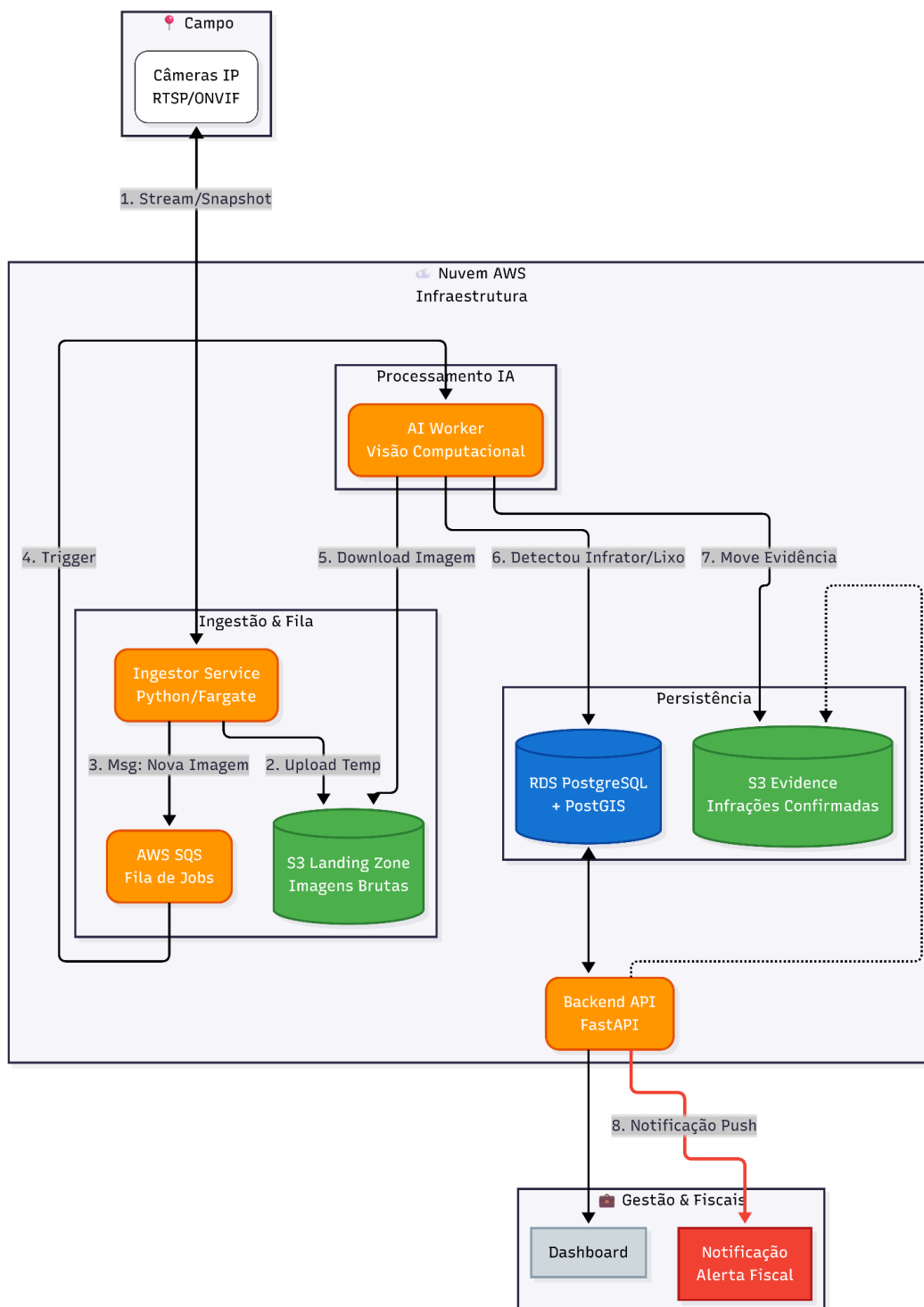
Esta estrutura foi concebida para ser flexível e escalável, suportando integralmente a transição para uma próxima fase de aceleração. O desenho arquitetural prevê, inclusive, uma futura migração para soluções baseadas em câmeras inteligentes (IA na borda), movimento que poderá ser realizado para otimizar os custos operacionais a longo prazo conforme a rede se expande.

Sob a ótica da engenharia, a solução fundamenta-se em uma arquitetura orientada a eventos e desacoplada, o que assegura a resiliência do sistema mesmo sob alta demanda. A utilização de filas assíncronas para orquestrar o fluxo entre a ingestão de imagens e o núcleo de inferência garante que os modelos de Deep Learning operem com estabilidade e precisão. Essa estrutura modular viabiliza a distinção técnica entre a acumulação passiva de resíduos e as infrações ativas e também consolida uma base de dados geoespacial estruturada e auditável, preparada para suportar a evolução contínua das regras de negócio.



2. Diagrama da solução

Abaixo, o fluxo de dados desde a captura das imagens no campo até a notificação do fiscal e subida dos dados ao dashboard.





3. Detalhamento dos componentes

a. Camada de ingestão

A camada de ingestão atua como o elo seguro entre o ambiente físico e a nuvem. O serviço conecta-se às câmeras IP utilizando protocolos padronizados de mercado, como o RTSP (*Real Time Streaming Protocol*, focado no transporte eficiente de dados de mídia) e o ONVIF (*Open Network Video Interface Forum*, padrão global que garante interoperabilidade entre diferentes fabricantes de segurança).

O sistema captura quadros estáticos (*snapshots*) em intervalos regulares, definidos de acordo com a necessidade do negócio (ex. 30s). Essa periodicidade é totalmente parametrizável; contudo, deve-se observar que o aumento da frequência de capturas escala proporcionalmente os custos de processamento e armazenamento na nuvem.

Para mitigar custos sem perder eficácia, o sistema suporta ajustes dinâmicos: ao detectar um potencial infrator ou anomalia, o intervalo de captura é drasticamente reduzido (aumentando a frequência) para garantir uma coleta rica de evidências. A arquitetura também já está preparada para, em uma fase futura, escalar para *streaming* de vídeo em tempo real apenas durante esses eventos críticos. Por fim, todas as imagens e metadados são transmitidos para o Data Lake (S3) através de canais criptografados via HTTPS/TLS, assegurando a integridade e confidencialidade dos dados desde a borda até o servidor.

b. Fila e desacoplamento

Para garantir a escalabilidade e resiliência do sistema, utilizamos o AWS SQS (*Simple Queue Service*) para gerenciar as filas de processamento da Inteligência Artificial. Este serviço atua como um *buffer* assíncrono, notificando o worker de IA assim que uma nova imagem está disponível para análise.

Essa arquitetura de desacoplamento assegura que todas as imagens sejam processadas na ordem correta de chegada e previne a formação de gargalos no backend. Mesmo em cenários de alta volumetria simultânea de coleta de imagem pelo ingestor, a fila regula a carga de trabalho entregue à IA, garantindo que o sistema de detecção opere com estabilidade e sem perda de dados.



c. Núcleo de inteligência

Este módulo atua como o coração do sistema SAÍRA, onde ocorre o processamento computacional intensivo das imagens recebidas. O cluster de IA consome as mensagens da fila e submete cada frame a modelos de redes neurais profundas (como YOLO v8/v11), treinados especificamente para identificar classes críticas como resíduos sólidos, entulho, pessoas e veículos.

A lógica de negócio embarcada é capaz de distinguir entre dois cenários operacionais distintos: a detecção de descarte irregular (constatação passiva de lixo acumulado no local) e o flagrante de Infração (identificação ativa de um indivíduo ou veículo no ato do descarte). Uma vez validada a detecção pela IA, o worker automaticamente persiste a imagem tratada no banco de evidências seguro e notifica o backend, que orquestra o registro da ocorrência ou o disparo de alertas críticos em tempo real.

d. Interfaces de usuário

O Dashboard Web, desenvolvido em React com TypeScript, atua como a camada de apresentação visual e estratégica do sistema. Ele consome os registros históricos e geoespaciais consolidados no banco de dados para gerar mapas de calor interativos e painéis de controle em tempo real, fornecendo aos gestores uma visão analítica precisa para o planejamento de rotas de limpeza e identificação de pontos viciados.

Paralelamente, o sistema conta com um módulo de notificações proativas destinado à mobilização das equipes de campo. Através de integração via API, o backend dispara alertas instantâneos assim que uma infração é detectada. Embora a implementação atual priorize o uso de mensageiros ágeis (como bots de Telegram) para garantir rapidez na resposta, a arquitetura foi desenhada para ser agnóstica ao canal, suportado nativamente a expansão para outros meios de comunicação, como e-mail, SMS ou integração com sistemas legados da prefeitura.



4. Infraestrutura de armazenamento e dados

A base da integridade do sistema reside em um banco de dados relacional. Diferente de planilhas simples ou bancos não-estruturados, este modelo organiza as informações em tabelas rígidas e interconectadas (linhas e colunas), garantindo que cada dado inserido respeite regras estritas de consistência e relacionamento. No contexto do SAÍRA, isso significa que uma ocorrência de lixo não pode existir sem estar vinculada a uma câmera válida e a um endereço real, além de outras informações intrínsecas aquele dado.

Para este projeto, adotamos uma estratégia de dados híbrida, segregando a informação em três repositórios lógicos principais:

- **Banco operacional e geoespacial (AWS RDS PostgreSQL):** Onde residem os dados estruturados ("quem", "quando", "onde"). É potencializado pela extensão PostGIS, permitindo que o sistema realize cálculos geográficos complexos nativamente, como verificar se um ponto de descarte está dentro da área de atuação de uma equipe de limpeza específica.
- **Repositório de evidências (Object Storage):** Armazenamento imutável das imagens que comprovam a infração.
- **Landing zone & dataset (Object Storage):** Área de trânsito para processamento bruto e retenção de dados para re-treinamento da IA.

Para evitar custos exponenciais, implementamos uma arquitetura de buckets segregada com políticas de ciclo de vida.



Abaixo, detalhamos a especificação técnica e a política de retenção de cada um.

Bucket / Pasta	Finalidade	Política de retenção (Lifecycle)
saira-landing-zone	Recebe todas as imagens brutas enviadas pelo Ingestor.	Exclusão automática: 24 horas. Se a IA não detectar infração, a imagem é descartada automaticamente.
saira-evidence-prod	Armazena apenas imagens confirmadas com infração (movidas pela IA).	Standard: 30 dias (Acesso rápido). Infrequent Access: após 30 dias. Glacier: após 1 ano (Auditoria).
saira-dataset	Amostras selecionadas para re-treinamento do modelo.	Retenção indefinida (volume controlado manualmente).

a. Perspectiva futura: armazenamento de vídeo

Considerando a possibilidade de expansão para captura de flagrantes em movimento, a arquitetura foi desenhada para suportar a inclusão de evidências em vídeo. Neste cenário, adotaríamos o padrão de "Armazenamento de Referência": arquivos binários pesados (vídeos) seriam depositados em um banco de dados S3 dedicado, enquanto o banco relacional armazenaria apenas o caminho do arquivo (URL) e seus metadados técnicos (duração, tamanho, codec).

No entanto, a implementação deste módulo envolve uma complexidade técnica elevada, exigindo uma camada adicional de transcodificação (converter vídeos brutos das câmeras para formatos compatíveis com web) e um aumento exponencial nos custos de storage e processamento. Portanto, esta funcionalidade é tratada como um item de roadmap futuro, a ser ativada apenas



se houver uma demanda crítica de negócio que justifique o investimento infraestrutural.

5. Stack tecnológico

a. Camada de software

A escolha tecnológica prioriza performance em processamento de imagens e agilidade no desenvolvimento.

- **Backend:** Python 3.10+ (Utilizando FastAPI para a API REST de alta performance e Scripts Python puros para os Workers de IA).
- **Inteligência artificial:** PyTorch e Ultralytics YOLO (v8/v11) para inferência de Visão Computacional.
- **Banco de dados:** PostgreSQL 15 com extensão PostGIS (para georreferenciamento avançado).
- **Mensageria:** AWS SQS (Gestão de filas assíncronas).
- **Frontend:** React com TypeScript.
- **Containerização:** Docker e Docker Compose.

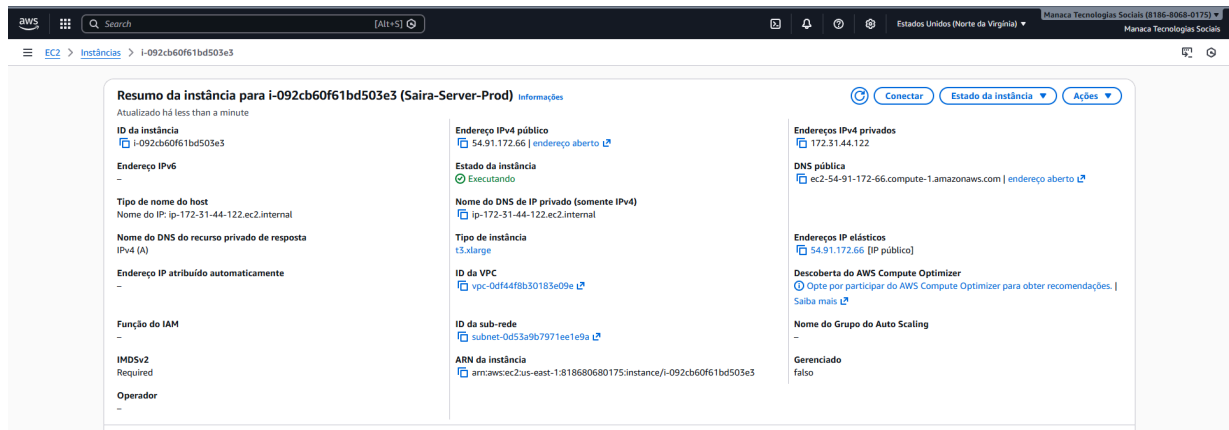
b. Especificação do servidor

Para o ambiente de produção inicial (MVP), a infraestrutura foi consolidada em uma instância de alta capacidade na AWS, dimensionada para equilibrar o consumo de memória da IA com custos operacionais.

- **Serviço de computação:** AWS EC2.
- **Tipo da Instância:** **t3.xlarge**.
 - **CPU:** 4 vCPUs (Intel Xeon – Família *Burstable*, ideal para cargas de trabalho variáveis como detecção de eventos).
 - **Memória RAM:** 16 GiB (Dimensionada para evitar *OOM Kills* durante o carregamento dos modelos de rede neural e operação do banco de dados).
- **Armazenamento:** 50 GB SSD (EBS gp3) de alta performance.
- **Sistema operacional:** Ubuntu Server 24.04 LTS.
- **Rede:** IP Estático Fixo (Elastic IP) para garantir conectividade ininterrupta com as câmeras e estabilidade das integrações.



Abaixo um print com máquina criada para o projeto:



6. Observabilidade e monitoramento

Para garantir a alta disponibilidade do sistema e a rápida detecção de falhas, a arquitetura implementa uma camada de observabilidade nativa integrada aos serviços da AWS.

- **Logs centralizados (AWS CloudWatch logs):** Todos os componentes (Ingestor, Workers de IA e API Backend) enviam seus logs de operação para grupos de logs estruturados. Isso permite a rastreabilidade completa de uma ocorrência, desde a captura da imagem até o alerta no Telegram, sem a necessidade de acesso SSH à máquina para auditoria.
- **Métricas de Infraestrutura:** O monitoramento de saúde do servidor (CPU, Memória RAM e Disco) e das filas SQS (número de mensagens aguardando processamento) é realizado via dashboards do CloudWatch. Alertas automáticos são configurados para notificar a equipe de sustentação caso a fila de processamento exceda limites seguros ou se o consumo de memória da instância atinja níveis críticos.

7. Segurança e controle de acesso

A segurança da arquitetura segue o princípio do privilégio mínimo e defesa em profundidade:



- **Gestão de identidade (IAM Roles):** A instância EC2 opera com uma *IAM Role* (Função) específica, que concede permissões estritas apenas para escrita nos buckets S3 e leitura da fila SQS. Dessa forma, **nenhuma credencial de acesso (access keys) é armazenada no código-fonte** ou nos arquivos de configuração do servidor, eliminando vetores de ataque comuns.
- **Segurança de rede (security groups):** O firewall virtual da AWS está configurado para bloquear todo o tráfego de entrada não essencial. O banco de dados PostgreSQL não possui IP público e aceita conexões exclusivamente da rede interna Docker, garantindo que os dados sensíveis permaneçam isolados da internet pública. A comunicação externa é restrita apenas às portas HTTP/HTTPS (para API) e SSH (para gestão, restrito a IPs confiáveis).

8. Estratégia de implantação (CI/CD)

O ciclo de vida de desenvolvimento adota práticas modernas de DevOps para garantir agilidade e estabilidade nas atualizações:

- **Versionamento:** Todo o código-fonte (Infraestrutura, IA e Backend) é versionado em repositório Git.
- **Containerização:** A aplicação é 100% baseada em Docker, garantindo paridade entre os ambientes de desenvolvimento e produção.
- **Implantação:** As atualizações na instância de produção são realizadas através da orquestração de containers, permitindo reversões rápidas em caso de falha e garantindo que as dependências do sistema (bibliotecas de IA, drivers) estejam sempre encapsuladas e testadas.